

L3 INFORMATIQUE

2025-2026

Simulateur de Réseau

RÉSEAUX — LE ROUTAGE

Rapport du TP4

CONTRIBUTEURS DU PROJET

Nathan ADOHO Gr A

Mise à jour : 4 mars 2026

Table des matières

1	Rappel du sujet	2
2	Analyse du sujet	2
2.1	Décomposition du problème	2
2.2	Choix d'architecture	2
3	Choix techniques effectués	3
3.1	Architecture générale : patron MVC	3
3.2	Intégration GraphStream	3
3.3	Modèle Java : classe ReseauLogique	4
3.4	Service d'interaction : ReseauInteractionService	4
3.5	Interface graphique : ItfResaux	5
4	Résultats et tests	5
4.1	Construction et manipulation du réseau	5
4.2	Calcul du plus court chemin (Dijkstra)	5
4.3	Table de routage	5
4.4	Chargement de fichier DGS	6
4.5	Contraintes métier	6
5	Conclusion	6
5.1	Bilan	6
5.2	Difficultés rencontrées	6
5.3	Perspectives	7
	Annexe : Mode d'emploi	8

1 Rappel du sujet

Ce TP porte sur l'implantation d'un programme Java permettant de simuler un réseau de commutateurs et de calculer le chemin le plus court dans une topologie réseau donnée. Le programme doit répondre aux exigences suivantes :

1. Permettre la saisie graphique du réseau (commutateurs, liens pondérés, machines terminales).
2. Permettre à l'utilisateur de sélectionner deux machines et de calculer l'ensemble des nœuds intermédiaires ainsi que les interfaces utilisées, via l'algorithme de Dijkstra.
3. Afficher le résultat sous forme graphique et textuelle.
4. Établir les tables de routage de chaque commutateur.

L'axe technique principal est la mise en œuvre d'une architecture modulaire en Java : un modèle de graphe exploitant GraphStream, piloté par une interface graphique riche développée en Java Swing.

2 Analyse du sujet

2.1 Décomposition du problème

Le sujet peut être décomposé en quatre grandes problématiques indépendantes mais reliées :

Modélisation du réseau Un réseau est un graphe pondéré non orienté. Les nœuds sont des commutateurs (switches) et des machines terminales. Les arêtes sont des liens pondérés entre commutateurs. Les machines terminales sont attachées aux commutateurs via des liaisons non pondérées (poids nul).

Représentation graphique interactive L'utilisateur doit pouvoir construire le réseau visuellement : ajouter/supprimer des commutateurs et des machines, créer des liens pondérés entre switches, charger une topologie depuis un fichier. La représentation doit être fidèle à la topologie réelle avec distinction visuelle claire entre switches et machines.

Calcul de plus court chemin L'algorithme de Dijkstra doit être appliqué entre deux machines quelconques. Le résultat doit être visualisé (coloration du chemin sur le graphe) et la distance totale affichée. Les tables de routage doivent être calculées pour chaque commutateur.

Circuit logique Pour une paire de machines donnée, il faut déterminer la séquence complète des commutateurs traversés, formant le circuit logique entre les deux terminaux.

2.2 Choix d'architecture

Une architecture modulaire en trois couches a été retenue : le modèle de graphe repose sur GraphStream (bibliothèque Java de manipulation et visualisation de graphes), l'interface graphique est développée en Java Swing, et un service d'interaction centralise la logique métier. Cette séparation respecte le patron MVC et facilite la maintenance.

3 Choix techniques effectués

3.1 Architecture générale : patron MVC

Le projet suit une architecture Modèle-Vue-Contrôleur stricte, organisée en classes Java distinctes :

- `TP4.ReseauLogique` : modèle du réseau, structure de graphe, algorithmes de routage.
- `TP4.ItfResaux` : vue Swing, organisation en onglets, gestion des composants graphiques.
- `TP4.ReseauInteractionService` : contrôleur, logique de traitement des événements utilisateur.
- `TP4.App` : point d'entrée de l'application.

Cette séparation permet de maintenir une cohérence forte entre les responsabilités : le modèle ne connaît pas l'interface, la vue délègue toute logique métier, et le service orchestre les interactions.

3.2 Intégration GraphStream

Structures de données GraphStream fournit une API de graphe complète avec gestion native des nœuds, arêtes, et attributs. Les principales structures utilisées sont :

- `Graph` : conteneur principal du réseau.
- `Node` : représente un commutateur ou une machine, avec attributs `ui.class`, `ui.label`, `ui.style`.
- `Edge` : lien entre deux nœuds, avec attribut `poid` (poids du lien).

Algorithme de Dijkstra via GraphStream L'implémentation utilise la classe `Dijkstra` fournie par GraphStream dans le package `org.graphstream.algorithm`. L'algorithme est exécuté depuis un nœud source avec la méthode `compute()`. La complexité est en $O((n + m) \log n)$ avec un tas de Fibonacci, adaptée aux graphes de taille moyenne.

Les fonctions principales sont :

- `dijkstra.compute()` : calcule les distances minimales depuis la source.
- `dijkstra.getPath(destination)` : reconstruit le chemin optimal sous forme de `Path`.
- `dijkstra.getPathLength(destination)` : retourne la distance totale.

Styles CSS pour le rendu GraphStream exploite un système de styles CSS pour le rendu visuel. Les propriétés critiques utilisées sont :

- `fill-color` : couleur de remplissage (nœuds, arêtes).
- `stroke-mode` : mode de tracé (`plain`, `dashes`).
- `size` : taille des arêtes (attention : pas `width`).
- `text-background-mode` : fond du texte de label (`none`, `rounded-box`).

Important : GraphStream utilise des noms de propriétés spécifiques incompatibles avec le CSS web standard. Par exemple, `stroke-mode: dashes` (et non `dashed`), `size` (et non `width`).

3.3 Modèle Java : classe `ReseauLogique`

Responsabilités `ReseauLogique` encapsule toute la logique du réseau :

- Gestion du graphe `GraphStream` (`graph`).
- Ajout/suppression de nœuds et de liens.
- Application des styles visuels (switches en bleu, machines en orange, liaisons machine-switch en pointillés).
- Calcul de plus court chemin avec Dijkstra.
- Calcul de table de routage pour un switch donné.
- Coloration du chemin calculé en vert.

Constantes métier Des constantes sont définies pour garantir la cohérence :

- `ATTR_POID` = "poid" : nom de l'attribut de poids sur les arêtes.
- `CLASS_SWITCH` = "switch" : classe CSS des commutateurs.
- `CLASS_MACHINE` = "machine" : classe CSS des machines.

Méthodes principales

- `ajouterSwitch(String id)` : ajoute un commutateur avec style bleu.
- `ajouterMachine(String id)` : ajoute une machine avec style orange.
- `lierSwitchs(String idA, String idB, int poids)` : crée une arête pondérée.
- `lierMachineSwitch(String idMachine, String idSwitch)` : liaison non pondérée en pointillés.
- `plusCourtChemin(String source, String destination)` : retourne un `Path` ou `null`.
- `colorierChemin(Path chemin)` : applique le style vert aux arêtes du chemin.
- `tableRoutage(String idSwitch)` : calcule les routes optimales depuis le switch.

Helpers

- `isSwitch(Node n)` : teste si un nœud est un switch.
- `isMachine(Node n)` : teste si un nœud est une machine.

3.4 Service d'interaction : `ReseauInteractionService`

Cette classe centralise toute la logique d'interaction utilisateur, déchargeant ainsi l'interface graphique. Elle maintient une référence au modèle (`ReseauLogique`) et déclenche un callback de rafraîchissement après chaque mutation.

Méthodes principales

- `ajouterSwitch()` : dialogue de saisie d'identifiant, appel au modèle, rafraîchissement.
- `ajouterMachine()` : dialogue de saisie, ajout au modèle.
- `lierDeuxSwitchs()` : sélection de deux switches, saisie du poids, création du lien.
- `supprimerNoeud()` : sélection d'un nœud, suppression avec confirmation.
- `afficherTableRoutage()` : sélection d'un switch, calcul de la table, affichage dans `JTable`.
- `afficherCircuitLogique()` : sélection de deux machines, calcul du chemin, affichage textuel et coloration graphique.
- `chargerFichier()` : ouverture d'un `FileChooser`, chargement d'un fichier DGS.

Le service utilise systématiquement `JOptionPane` pour les dialogues et `JTable/ScrollPane` pour les affichages tabulaires.

3.5 Interface graphique : ItfResaux

Organisation en onglets L'interface est structurée avec un `JTabbedPane` contenant trois onglets :

- **Fichier** : chargement de topologies DGS.
- **Outils** : ajout de switches, machines, liaison, suppression.
- **Analyse** : table de routage, circuit logique.

Composants principaux

- **Viewer** (`GraphStream`) : surface de rendu du graphe, intégrée dans un `JPanel`.
- **JButton** : boutons d'action connectés aux méthodes du service.
- **JComboBox** : sélecteurs de nœuds filtrés par type (switches ou machines).

Callback de rafraîchissement Après chaque action métier, le service appelle `rafraichirInterface` qui met à jour les listes déroulantes et force un repaint du viewer.

4 Résultats et tests

4.1 Construction et manipulation du réseau

L'interface permet de créer un réseau de commutateurs et de machines depuis l'onglet Outils. Les switches et machines sont ajoutés via des dialogues de saisie. Les liaisons entre switches sont pondérées (saisie d'un poids entier positif). Les liaisons machine-switch sont automatiques avec poids nul et rendu en pointillés.

4.2 Calcul du plus court chemin (Dijkstra)

Le calcul est effectif entre deux machines quelconques depuis l'onglet Analyse. Le résultat s'affiche sous forme textuelle (liste ordonnée des commutateurs traversés) et graphiquement (arêtes colorées en vert). La visualisation est claire : le chemin est mis en évidence avec style distinct (`fill-color: green, size: 4px`).

Test de cohérence Sur un réseau à 4 commutateurs (S1, S2, S3, S4) avec topologie simple, le chemin calculé correspond au chemin optimal attendu. Par exemple, pour un réseau linéaire S1–S2–S3–S4 avec poids uniformes, le chemin de M1 (attachée à S1) vers M4 (attachée à S4) traverse S1, S2, S3, S4 avec distance totale égale à la somme des poids.

4.3 Table de routage

La table de routage s'affiche correctement pour chaque switch sélectionné. Le tableau comporte trois colonnes :

- **Destination** : identifiant du nœud de destination.
- **Via** : identifiant du prochain saut (voisin direct).
- **Coût** : distance totale depuis le switch source.

Validation Pour un réseau simple avec S1 relié à S2 (poids 10) et S2 relié à S3 (poids 5), la table de routage de S1 contient :

- Destination = S2, Via = S2, Coût = 10
- Destination = S3, Via = S2, Coût = 15

4.4 Chargement de fichier DGS

Le chargement depuis un fichier DGS fonctionne correctement. Le graphe est reconstitué avec tous ses nœuds, arêtes, attributs et styles. Le viewer GraphStream affiche immédiatement la topologie chargée.

4.5 Contraintes métier

Les contraintes suivantes sont respectées :

- Une machine ne peut pas être reliée directement à une autre machine.
- Une machine est reliée à un seul switch.
- Les liaisons machine-switch ont un poids nul et sont rendues en pointillés.

5 Conclusion

5.1 Bilan

Ce projet a permis de mettre en œuvre un simulateur de réseau complet répondant à toutes les exigences du sujet. L'architecture MVC appliquée côté Java rend le code modulaire et facilement extensible. Le patron de séparation entre modèle, vue et service garantit une maintenance aisée et une évolution progressive des fonctionnalités.

Les fonctionnalités implémentées couvrent l'intégralité des points obligatoires (saisie graphique, Dijkstra, tables de routage) ainsi que des extensions utiles (machines terminales, circuit logique, chargement de fichiers DGS).

5.2 Difficultés rencontrées

Propriétés CSS GraphStream GraphStream utilise un système de propriétés CSS spécifique, incompatible avec le CSS web standard. Les erreurs fréquentes incluent :

- Utilisation de `width` au lieu de `size` pour les arêtes.
- Utilisation de `stroke-mode: dashed` au lieu de `stroke-mode: dashes`.
- Valeurs invalides pour `text-background-mode`.

La documentation GraphStream est parfois imprécise, nécessitant des tests empiriques pour valider les styles.

Gestion des ports de switch La gestion manuelle des numéros de port sur chaque switch s'est révélée complexe dans le cadre de cette interface graphique. Un mécanisme d'auto-incrémentation implicite a été retenu comme compromis pragmatique. L'exposition explicite des ports dans l'interface (numérotation visible, configuration manuelle) constitue une amélioration possible pour une version ultérieure.

Dettes techniques initiales La classe d'interface initiale (`ItfResaux`) contenait plus de 500 lignes, mélangeant logique d'affichage et logique métier. La refactorisation avec extraction de `ReseauInteractionService` a permis de réduire cette taille à environ 200 lignes, améliorant significativement la lisibilité et la testabilité.

5.3 Perspectives

- **Export et sauvegarde** : ajout d'une fonctionnalité d'export au format DGS pour persister les topologies créées.
- **Undo/Redo** : implémentation d'un historique d'actions avec possibilité d'annulation.
- **Configuration manuelle des ports** : interface dédiée à la gestion fine des numéros de port sur chaque switch.
- **Tests unitaires** : couverture du modèle avec JUnit pour garantir la robustesse des algorithmes.
- **Simulation de panne** : ajout d'un mécanisme de désactivation temporaire de switches pour tester la résilience du routage.

ANNEXE

MODE D'EMPLOI DE L'INTERFACE

Guide complet d'utilisation du simulateur
de réseau avec routage et circuit logique.

Simulateur de routage — L3 Informatique 2025–2026

A.1 — Vue générale de l'application

L'interface principale est organisée en onglets regroupant les fonctionnalités par thème :

- **Onglet Fichier** : chargement de topologies depuis des fichiers DGS.
- **Onglet Outils** : construction et modification du réseau (ajout de switches, machines, liaisons, suppressions).
- **Onglet Analyse** : calcul de table de routage et affichage du circuit logique entre deux machines.

Le graphe est affiché dans une surface de visualisation centrale (viewer GraphStream) où les nœuds et arêtes sont rendus en temps réel.

A.2 — Construction du réseau

Ajouter un commutateur

1. Se placer dans l'onglet **Outils**.
2. Cliquer sur le bouton **Ajouter Switch**.
3. Saisir un identifiant unique dans la boîte de dialogue (ex : **S1**).
4. Valider : le switch apparaît dans le graphe avec un rendu bleu circulaire.

Ajouter une machine

1. Dans l'onglet **Outils**, cliquer sur **Ajouter Machine**.
2. Saisir un identifiant unique (ex : **M1**).
3. Saisir l'identifiant du switch auquel rattacher la machine.
4. Valider : la machine apparaît en orange rectangulaire, reliée au switch par une liaison en pointillés.

Relier deux commutateurs

1. Cliquer sur **Lier deux Switchs**.
2. Saisir l'identifiant du premier switch (ex : **S1**).
3. Saisir l'identifiant du second switch (ex : **S2**).
4. Saisir le poids du lien (entier positif, ex : 10).
5. Valider : un lien plein est tracé entre les deux switches avec le poids affiché.

Supprimer un nœud

1. Cliquer sur **Supprimer Noeud**.
2. Saisir l'identifiant du nœud à supprimer (switch ou machine).
3. Confirmer : le nœud et toutes ses arêtes sont supprimés du graphe.

A.3 — Chargement d'une topologie

1. Se placer dans l'onglet **Fichier**.
2. Cliquer sur **Charger fichier DGS**.
3. Sélectionner un fichier **.dgs** valide dans l'explorateur.
4. Le graphe est reconstruit automatiquement avec tous les nœuds, arêtes et attributs.

Note : le format DGS est un format texte décrivant un graphe (nœuds, arêtes, attributs). GraphStream le supporte nativement.

A.4 — Calcul de la table de routage

1. Se placer dans l'onglet **Analyse**.
2. Cliquer sur **Table de Routage**.
3. Sélectionner un switch dans la liste déroulante.
4. Valider : une fenêtre s'ouvre affichant un tableau avec trois colonnes : **Destination**, **Via**, **Coût**.

Le tableau liste toutes les destinations accessibles depuis le switch sélectionné, avec le prochain saut optimal et la distance totale.

A.5 — Affichage du circuit logique

Le circuit logique représente le chemin complet entre deux machines, sous forme textuelle et graphique.

1. Dans l'onglet **Analyse**, cliquer sur **Circuit Logique**.
2. Sélectionner la machine source dans la première liste déroulante.
3. Sélectionner la machine destination dans la seconde liste déroulante.
4. Valider :
 - Le chemin est affiché dans une boîte de dialogue (liste ordonnée des commutateurs traversés).
 - Le chemin est coloré en **vert** sur le graphe pour identification visuelle immédiate.

Si aucun chemin n'existe (réseau déconnecté), un message d'erreur est affiché.

A.6 — Lecture du graphe affiché

Représentation visuelle

- **Commutateurs** : cercles de couleur **bleue**.
- **Machines** : rectangles de couleur **orange**.
- **Liens entre switches** : lignes pleines avec poids affiché (étiquette).
- **Liens machine-switch** : lignes en **pointillés** sans poids.
- **Chemin calculé** : arêtes en **vert épais** après calcul de circuit logique.

Interprétation La topologie affichée reflète exactement l'état du modèle internal. Chaque mutation (ajout, suppression, liaison) est immédiatement visible dans le viewer.

A.7 — Compilation et exécution

Compiler le projet Depuis la racine du projet (où se trouve `pom.xml`), exécuter :

```
mvn clean compile
```

Exécuter l'application Depuis la racine du projet :

```
mvn exec:java -Dexec.mainClass="TP4.App"
```

Ou bien, si les classes sont déjà compilées :

```
java -cp target/classes TP4.App
```